TECHNICAL REPORT

Post-Quantum Pretty Good Privacy (PQ-PGP)

A COMPOSITE POST-QUANTUM OPENPGP IMPLEMENTATION

RFC 9980 · FIPS 203 (ML-KEM) · FIPS 204 (ML-DSA) · RFC 7748 / 8032 · RFC 5869 · NIST SP 800-38D / 38F · RFC 7253 · RFC 9106

October 2026

ABSTRACT

This report documents the design, operation, and security analysis of Post-Quantum Pretty Good Privacy (PQ-PGP), a self-contained browser application that implements composite post-quantum OpenPGP cryptography in accordance with RFC 9980. The application pairs a NIST-standardised lattice-based primitive (ML-KEM from FIPS 203, ML-DSA from FIPS 204) with a classical elliptic-curve primitive (X25519 or X448 for ECDH, Ed25519 or Ed448 for EdDSA) at each cryptographic operation, deriving the wrapping key from both legs via the HKDF-SHA256 combiner specified in RFC 9980.

We describe the three high-level operations --- key generation, encrypt-and-sign, and decrypt-and-verify --- with block-flow diagrams, algorithm pseudocode, byte-layout tables for every on-wire packet format, and a structured threat model covering passive and active network adversaries, classical offline dictionary attacks, quantum adversaries with Shor's algorithm, and the harvest-now-decrypt-later scenario. Every standard cited in the application's user interface is verified against the executed code; the citation accuracy was confirmed by direct inspection. The report concludes with a discussion of in-scope guarantees, known limitations, and future work.

KEYWORDS post-quantum cryptography · ML-KEM · ML-DSA · RFC 9980 · OpenPGP · composite KEM combiner · AES-KW · HKDF-SHA256 · Argon2id · AES-OCB3 · browser cryptography

Contents

1 Introduction **4**

2 Standards and Algorithms Reference **4**

3 System Architecture **6**

4 Composite Variant Selection **7**

5 Key Generation Operation **7**

5.1 Block Flow . . . 9

5.2 Algorithm Pseudocode . . . 9

5.3 Security notes . . . 9

6 Encryption + Signing Operation **10**

6.1 Block Flow . . . 10

6.2 Algorithm Pseudocode . . . 11

7 Decryption + Verification Operation **11**

7.1 Block Flow . . . 12

7.2 Algorithm Pseudocode . . . 12

8 Packet Formats and ASCII Armor **13**

9 Cryptographic Algorithms in Depth **14**

9.1 ML-KEM (FIPS 203) . . . 14

9.2 ML-DSA (FIPS 204) . . . 15

9.3 X25519 and X448 (RFC 7748) . . . 15

9.4 Ed25519 and Ed448 (RFC 8032) . . . 15

9.5 HKDF-SHA256 (RFC 5869) . . . 15

9.6 AES Key Wrap and AES Key Wrap with Padding (NIST SP 800-38F, RFC 3394/5649) . . . 15

9.7 AES-256-GCM (NIST SP 800-38D) . . . 16

9.8 AES-256-OCB3 (RFC 7253) . . . 16

9.9 AES-256-EAX (RFC 9580 §5.13.4) . . . 16

9.10 Argon2id (RFC 9106) . . . 17

10 Security Analysis and Threat Model **17**

10.1 Adversary model . . . 17

10.2 In-scope guarantees . . . 18

10.3 Mitigations . . . 18

10.4 Out-of-scope and limitations . . . 18

10.5 Cipher selection notes 19
10.6 At-rest protection notes 19

11 Conclusion **19**

List of Figures

1 System architecture overview. User input flows through the UI into the in-memory state object; the crypto core orchestrates primitive calls to the Web Crypto API and the ESM-bundled `@noble` libraries; the result is serialized to an ASCII-armored block and exposed back to the user. 6

2 Key-generation block flow. Four key components are generated independently, then serialised into a public key block and a password-sealed private key block. The fingerprint is derived from the four public keys via SHA-256. 8

3 Encrypt-and-sign block flow. Three parallel input streams converge through KEM encapsulation, KEK derivation, AES-KW session-key wrap, and AEAD payload encryption; if signing is enabled, the plaintext bytes are also fed through composite DSA signing before final message-block serialisation. 10

4 Decrypt-and-verify block flow. The recipient unlocks the private key, decodes the message block, re-runs KEM decapsulation and ECDH, re-derives the KEK, unwraps the session key (whose AES-KW integrity check transitively verifies that both KEM legs produced the correct shared secrets), then trial-decrypts the AEAD ciphertext. If the message is signed, both signature components are verified against the recovered plaintext. 12

List of Tables

1 Standards cited in the UI and the corresponding code construct. 5

2 Composite variant parameters. All sizes in bytes. 7

3 PQ-PGP message block byte layout (v1). All multi-byte integers are big-endian. 13

4 PQ-PGP V3 secret-key block byte layout (current format). All multi-byte integers are big-endian. V2 blocks replace the four `argon2*` fields with a single 12-byte `iv` field and use AES-256-GCM instead of AES-256-KWP for the `wrappedBlob`. . 14

1 Introduction

The advent of large-scale quantum computers threatens the classical public-key cryptography on which contemporary secure messaging rests. Shor's algorithm, when run on a sufficiently large quantum machine, factors RSA moduli and computes discrete logarithms over elliptic curves in polynomial time, breaking RSA, DH, ECDH, ECDSA, and EdDSA outright. Even before such machines exist, the *harvest-now-decrypt-later* adversary can record long-lived ciphertexts today and decrypt them retroactively once a quantum adversary becomes available. This realisation has driven NIST's Post-Quantum Cryptography (PQC) standardisation effort, which in 2024 produced FIPS 203 (ML-KEM, formerly CRYSTALS-Kyber) and FIPS 204 (ML-DSA, formerly CRYSTALS-Dilithium) as the first U.S. federal lattice-based standards.

The IETF OpenPGP working group has responded with *RFC 9980*, the composite-KEM combiner specification that mandates ML-KEM always be paired with a classical ECDH curve, so that the resulting hybrid resists both quantum and classical attacks simultaneously. PQ-PGP, the application analysed in this report, is a self-contained browser implementation of that composite model. It is a single-page HTML application that performs key generation, authenticated encryption, and signature verification in the browser, with no server-side component. All cryptographic primitives are provided by audited JavaScript libraries: `@noble/post-quantum` for ML-KEM and ML-DSA, `@noble/curves` for X25519/X448 and Ed25519/Ed448, `@noble/ciphers` for AES, and `hash-wasm` for Argon2id.

The purpose of this report is to document, in technical depth, exactly how the application operates, which standards it implements, what algorithms it employs, and what security guarantees it provides. We include block-flow diagrams of every operation, byte-layout tables for every on-wire format, pseudocode for every cryptographic primitive, and a structured threat model. The report is intended both as an implementation reference for engineers wishing to interoperate with PQ-PGP, and as a security review artefact for auditors verifying that the displayed standard citations match the executed code.

2 Standards and Algorithms Reference

Table 1 enumerates every standard cited in the application's user interface and maps it to the specific code construct that implements it. Each row was verified by direct inspection of the source code; the citation accuracy was confirmed in a separate audit. The set of standards spans four broad families: post-quantum primitives (FIPS 203/204), classical elliptic-curve primitives (RFC 7748/8032), a combiner/KDF layer (RFC 9980/5869), and symmetric primitives for key wrapping, AEAD, and password hashing (NIST SP 800-38D/F, RFC 7253, RFC 9106).

The standards are layered in a deliberate order: ML-KEM and ML-DSA provide the quantum-resistant backbone, classical curves provide a defense-in-depth leg that remains useful even if a flaw is later discovered in the lattice schemes, the RFC 9980 combiner cryptographically binds the two KEM legs together so that an attacker must break *both* to recover the KEK, AES-KW adds an integrity check that doubles as a session-key authenticity oracle, and the AEAD layer provides confidentiality and integrity for the payload itself. The Argon2id KDF is reserved for offline password-to-key derivation on the private key block, where its memory hardness is the only thing standing between a stolen key block and a successful dictionary attack.

Table 1: Standards cited in the UI and the corresponding code construct.

Standard	**Purpose in PQ-PGP**	**Code construct**
FIPS 203 (ML-KEM)	Post-quantum IND-CCA2 KEM, encapsulation side	`ml_kem768`/`ml_kem1024` from `@noble/post-quantum`
FIPS 204 (ML-DSA)	Post-quantum EUF-CMA signature	`ml_dsa65`/`ml_dsa87` from `@noble/post-quantum`
RFC 7748	X25519 / X448 ECDH (classical KEM leg)	`x25519`, `x448` from `@noble/curves`
RFC 8032	Ed25519 / Ed448 EdDSA (classical signature leg)	`ed25519`, `ed448` from `@noble/curves`
RFC 9980	Composite KEM combiner (HKDF over PQ + classical shared secrets)	`deriveKekRfc9980()`, `buildKemCombinerInfo()`
RFC 5869	HKDF-SHA256 (extract-and-expand)	`hkdfSha256()` via Web Crypto `deriveBits`
NIST SP 800-38F	AES Key Wrap (RFC 3394 KW) for session-key wrapping (32-byte aligned data)	`aesKwWrap()`, `aesKwUnwrap()` via Web Crypto `wrapKey`/`unwrapKey`
RFC 5649	AES Key Wrap with Padding (KWP) for arbitrary-length private-key-block wrapping	`aesKwpWrap()`, `aesKwpUnwrap()` — manual AES-KW W algorithm over AES-ECB from `@noble/ciphers`
NIST SP 800-38D	AES-256-GCM AEAD (default data cipher)	`aesGcmEncrypt()`/`aesGcmDecrypt()` via Web Crypto
RFC 7253	AES-256-OCB3 AEAD (alternative data cipher)	`aesOcbEncrypt()`/`aesOcbDecrypt()` pure-JS implementation
RFC 9580 §5.13.4	AES-256-EAX AEAD (third data cipher, formally proven)	`aesEaxEncrypt()`/`aesEaxDecrypt()` pure-JS implementation over AES-ECB
RFC 9106	Argon2id memory-hard KDF for private-key-block encryption (parameters encoded in S2K packet per RFC 9580 §3.7.1.4)	`deriveKeyFromPassword()` via `hash-wasm`; params read from packet on decode

3 System Architecture

PQ-PGP is delivered as a single self-contained HTML file with an embedded ES module. The user interface is divided into three tabs — *Key Generation*, *Encrypt (+Sign)*, and *Decrypt (+Verify)* — each of which corresponds to one of the three high-level cryptographic operations. A per-tab variant selector (MUST vs. SHOULD) is stored in a single global `STATE` object, together with mode-specific bookkeeping such as file-upload state, cipher choice, and decoded message bytes. All cryptographic state lives only in browser memory; nothing is persisted to disk or transmitted over the network. The user can copy, download, or upload armored text blocks at any time, and the application converts between byte arrays and armored text on the fly using a CRC-24-protected ASCII armor.

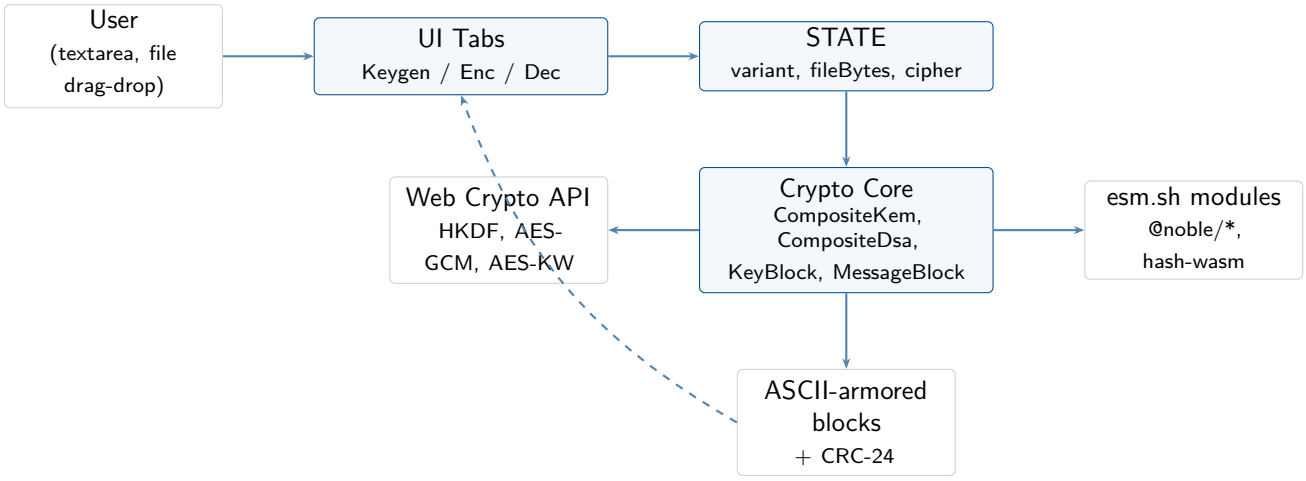


Figure 1: System architecture overview. User input flows through the UI into the in-memory state object; the crypto core orchestrates primitive calls to the Web Crypto API and the ESM-bundled `@noble` libraries; the result is serialized to an ASCII-armored block and exposed back to the user.

The cryptographic core is organised around four logical modules. `CompositeKem` performs keypair generation, encapsulation, and decapsulation for the KEM layer, internally invoking both ML-KEM and the classical ECDH primitive in sequence. `CompositeDsa` performs the analogous role for the signature layer, pairing ML-DSA with EdDSA. `KeyBlock` handles serialisation, deserialisation, and password-based encryption of the composite key material, including the Argon2id KDF and the AES-256-GCM seal over the inner secret bytes. `MessageBlock` handles the on-wire message format, including the optional signature fields and the file-payload flag. The AEAD layer (`dataEncrypt`, `dataDecryptTrial`) is policy code that selects between AES-256-GCM and AES-256-OCB3 and, on decryption, performs a trial-decrypt sequence that recovers the cipher actually used by the sender.

All long-lived secrets — the four private key components, the session key, the KEK — are stored as `Uint8Array` instances and are zeroised only by JavaScript garbage collection. The application does not currently call `.fill(0)` on secret buffers before they go out of scope; this is a known limitation discussed in Section 10. The Argon2id parameters are fixed at `parallelism=1`, `iterations=4`, `memorySize=262144` KiB (256 MiB), `hashLength=32`; these are chosen to make each brute-force attempt cost roughly one CPU-second on a 2024-era laptop while still completing in under one second for the legitimate user. The salt is a fresh 16-byte random value generated per key block. The private-key block is sealed with AES-256-KWP (NIST SP 800-38F Key Wrap with Padding, RFC 5649) rather than AES-256-GCM; this eliminates the need for a per-encryption IV, because AES-KW is deterministic. The Argon2id parameters are encoded in the S2K packet immediately after the salt (per RFC 9580 §3.7.1.4), so a future change to

those values remains backward-compatible. The AEAD layer supports three ciphers (AES-256-GCM, AES-256-OCB3, AES-256-EAX); the choice is signalled by the user at encryption time and recovered by trial decryption on the receiver side.

4 Composite Variant Selection

The application exposes two pre-defined composite variants, both of which combine an ML-KEM KEM with a classical ECDH curve and an ML-DSA signature with a classical EdDSA curve. The variant is selected per-tab via a pair of clickable cards, and the selection is stored in `STATE.variant[tabKey]`. The variant identifier is also encoded as the sixth byte of every PQ-PGP block (immediately after the four-byte magic and the one-byte version), so that a mismatch between the recipient's key and the loaded message is detected structurally before any cryptographic work is performed. The application refuses to mix variants: if a recipient public key of variant MUST is loaded into an encrypt tab set to variant SHOULD, an explicit error is raised and the operation aborts before KEM encapsulation.

Table 2: Composite variant parameters. All sizes in bytes.

	MUST (baseline, Cat-3)	**SHOULD (hardened, Cat-5)**
Variant ID (byte 5)	0x01	0x02
KEM (PQ)	ML-KEM-768 (`ml_kem768`)	ML-KEM-1024 (`ml_kem1024`)
pk / sk / ct / ss	1184 / 2400 / 1088 / 32	1568 / 3168 / 1568 / 32
KEM (classical)	X25519	X448
pk / sk / ss	32 / 32 / 32	56 / 56 / 56
DSA (PQ)	ML-DSA-65 (`ml_dsa65`)	ML-DSA-87 (`ml_dsa87`)
pk / sk / sig	1952 / 4032 / 3309	2592 / 4896 / 4627
DSA (classical)	Ed25519	Ed448
pk / sk / sig	32 / 32 / 64	57 / 57 / 114
NIST PQC category	3 (≈ AES-192)	5 (≈ AES-256)

The variant coupling rule — ML-KEM strength tied to ML-DSA strength, and PQ strength tied to classical strength — is enforced by the structure of the `VARIANTS` table itself: each variant is a single object that bundles both the KEM pair and the DSA pair, and the application selects the entire object atomically. There is no API path that allows, for example, ML-KEM-768 paired with ML-DSA-87, because doing so would mix security categories and undermine the simpler security argument. The fingerprint algorithm (Section 5) hashes the four public keys together with SHA-256, so any cross-variant key substitution would change the fingerprint and be detectable out-of-band.

5 Key Generation Operation

The key-generation flow produces two ASCII-armored blocks: a public key block (which can be freely shared) and a private key block (which is password-protected and must be kept secret by its owner). The user supplies an optional identity string and a mandatory password; the application generates four independent keypairs — ML-KEM, classical ECDH, ML-DSA, classical EdDSA — and serialises them with the RFC 9980 variant tag.

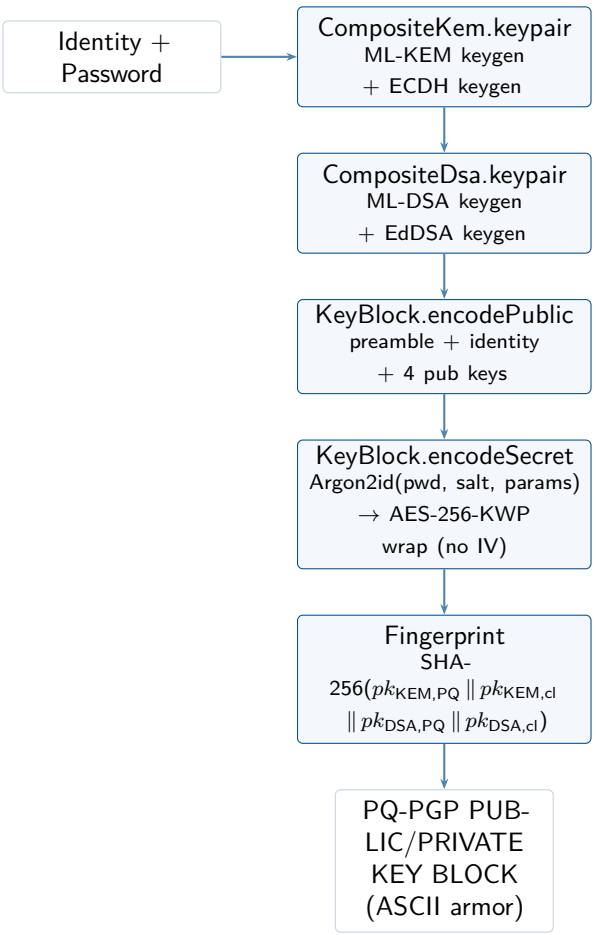


Figure 2: Key-generation block flow. Four key components are generated independently, then serialised into a public key block and a password-sealed private key block. The fingerprint is derived from the four public keys via SHA-256.

5.1 Block Flow

5.2 Algorithm Pseudocode

The composite keypair generation simply calls the underlying primitive keygens:

Algorithm 1: CompositeKem.keypair and CompositeDsa.keypair (combined)

Input: Variant $v \in \{\text{MUST}, \text{SHOULD}\}$
Output: Composite keypair *kp*

$pqKp_{\text{KEM}} \leftarrow v.\text{KEM.PQ.impl.keygen}()$;
$sk_{\text{cl,KEM}} \leftarrow v.\text{KEM.cl.utils.randomPrivateKey}()$;
$pk_{\text{cl,KEM}} \leftarrow v.\text{KEM.cl.getPublicKey}(sk_{\text{cl,KEM}})$;
$pqKp_{\text{DSA}} \leftarrow v.\text{DSA.PQ.impl.keygen}()$;
$sk_{\text{cl,DSA}} \leftarrow v.\text{DSA.cl.utils.randomPrivateKey}()$;
$pk_{\text{cl,DSA}} \leftarrow v.\text{DSA.cl.getPublicKey}(sk_{\text{cl,DSA}})$;
return $(pqKp_{KEM}, sk_{cl,KEM}, pk_{cl,KEM},$
  $pqKp_{DSA}, sk_{cl,DSA}, pk_{cl,DSA})$;

The private key block is sealed under a key derived from the user password via Argon2id, then AES-256-GCM-encrypted with a fresh random IV:

Algorithm 2: KeyBlock.encodeSecret (V3 password-protected, AES-256-KWP + Argon2 params in S2K)

Input: Keypair *kp*, variant v, identity *id*, password *pwd*
Output: Secret-key block bytes *secBytes* (V3 format)

$inner \leftarrow \text{LP}(sk_{\text{KEM,PQ}}) \,\|\, \text{LP}(sk_{\text{KEM,cl}})$;
$inner \leftarrow inner \,\|\, \text{LP}(sk_{\text{DSA,PQ}}) \,\|\, \text{LP}(sk_{\text{DSA,cl}})$;
`// lp = length-prefixed encoding: u32(len) || bytes`
$salt \xleftarrow{\$} \{0,1\}^{128}$; `// 16 bytes`
`// Argon2id S2K parameters (RFC 9106), encoded in the S2K packet per RFC 9580 §3.7.1.4`
$params \leftarrow (p{=}1,\ t{=}4,\ m{=}262144 \text{ KiB},\ \ell{=}32)$;
$dk \leftarrow \text{Argon2id}(pwd, salt, params)$;
`// No IV: AES Key Wrap (NIST SP 800-38F) is deterministic`
$wrappedBlob \leftarrow$ AES-256-KWP-Wrap$(dk, inner)$; `// RFC 5649: MLI-prefixed, zero-padded, then 6-round W`
$body \leftarrow \text{MAGIC}_{\text{secret}} \,\|\, \mathbf{0x03} \,\|\, v.\text{id} \,\|\, \text{LP}(id)$;
$body \leftarrow body \,\|\, salt \,\|\, \text{U32}(p) \,\|\, \text{U32}(t) \,\|\, \text{U32}(m) \,\|\, \text{U32}(\ell)$;
$body \leftarrow body \,\|\, \text{LP}(wrappedBlob)$;
$body \leftarrow body \,\|\, \text{LP}(pk_{\text{KEM,PQ}}) \,\|\, \text{LP}(pk_{\text{KEM,cl}})$;
$body \leftarrow body \,\|\, \text{LP}(pk_{\text{DSA,PQ}}) \,\|\, \text{LP}(pk_{\text{DSA,cl}})$;
return *body*;

The public-key fingerprint is computed as a SHA-256 hash of the concatenation of the four public keys, formatted as eight groups of four hex bytes separated by single and double spaces (OpenPGP-v6 style). This format is deterministic, collision-resistant under SHA-256, and tamper-evident: any substitution of any of the four public-key components produces a completely different fingerprint. The fingerprint is displayed to the user at key-generation time and again when a public key is loaded into the encrypt tab, so two parties can verify out-of-band that they hold the same composite public key.

5.3 Security notes

The private key block version byte distinguishes two formats: `0x02` (V2, legacy, AES-256-GCM with IV) and `0x03` (V3, current, AES-256-KWP with Argon2 params encoded in the S2K packet).

The decoder accepts both versions, so V2 key blocks produced by older versions of the application remain decryptable. V3 blocks signal three things to the decoder: (i) the Argon2id parameters are stored in the S2K packet immediately after the salt, so the decoder does not need to rely on hardcoded constants; (ii) the inner private-key material is wrapped with AES-256-KWP (NIST SP 800-38F Key Wrap with Padding, RFC 5649), which is deterministic and therefore does not require an IV; and (iii) the wrapped blob carries an implicit integrity check via the AES-KW W algorithm's constant 0xA6A6A6A6A6A6A6A6 initial value, so a wrong KEK (from a wrong password) is detected at unwrap time with a clear error. The public keys are also stored unencrypted inside the secret-key block, so the recipient can compute the fingerprint even without unlocking the inner secret — this lets the application display key metadata in the encrypt-tab sender-privkey panel before asking for the password.

6 Encryption + Signing Operation

The encrypt-and-sign flow takes three inputs: a recipient public key block, an optional sender private key block (with password), and a plaintext (either text or file). It produces a single ASCII-armored PQ-PGP message block that contains the ciphertext, the wrapped session key, both KEM ciphertexts, and (if signing is enabled) both signature components.

6.1 Block Flow

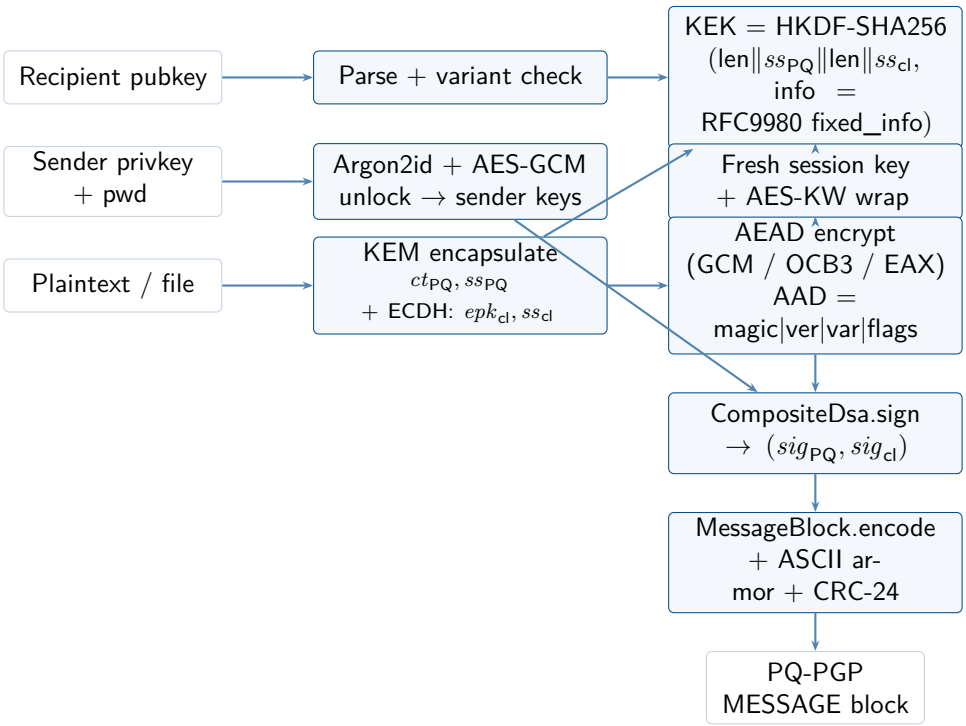


Figure 3: Encrypt-and-sign block flow. Three parallel input streams converge through KEM encapsulation, KEK derivation, AES-KW session-key wrap, and AEAD payload encryption; if signing is enabled, the plaintext bytes are also fed through composite DSA signing before final message-block serialisation.

6.2 Algorithm Pseudocode

KEM encapsulation produces two shared secrets that are then combined via the RFC 9980 HKDF combiner:

Algorithm 3: CompositeKem.encapsulate

Input: Recipient composite public key pk_R, variant v
Output: $(ct_{\text{PQ}}, epk_{\text{cl}}, wrappedSK, sessionKey)$

$(ct_{\text{PQ}}, ss_{\text{PQ}}) \leftarrow v.\text{KEM.PQ.impl.encapsulate}(pk_R.\text{PQ})$;
$esk_{\text{cl}} \xleftarrow{\$} \{0,1\}^{256}$; `// random ephemeral secret`
$epk_{\text{cl}} \leftarrow v.\text{KEM.cl.impl.getPublicKey}(esk_{\text{cl}})$;
$ss_{\text{cl}} \leftarrow v.\text{KEM.cl.impl.getSharedSecret}(esk_{\text{cl}}, pk_R.\text{cl})$;
$ikm \leftarrow \text{LP}(ss_{\text{PQ}}) \,\|\, \text{LP}(ss_{\text{cl}})$;
$info \leftarrow \text{BUILDINFO}(v, pk_R)$;
$kek \leftarrow \text{HKDF-SHA256}(ikm, \epsilon, info, 32)$;
$sessionKey \xleftarrow{\$} \{0,1\}^{256}$;
$wrappedSK \leftarrow \text{AES-KW-WRAP}(kek, sessionKey)$;
return $(ct_{PQ}, epk_{cl}, wrappedSK, sessionKey)$;

The `buildInfo` function constructs the RFC 9980 fixed-info structure as a length-prefixed concatenation of the version byte, the domain-separation tag `OpenPGPCompositeKDFv1`, the two algorithm name strings (e.g. `ML-KEM-768` and `X25519`), the two recipient public keys, and the output length. This binds the KEK to the specific algorithm choices and to the recipient's identity, preventing cross-protocol and cross-key attacks. The HKDF salt is set to the empty string, which is the RFC 9980 default; the salt-like role is played by the info field instead.

Composite signing signs the plaintext bytes (after file-payload framing, if applicable) with both signature schemes and returns both signatures for inclusion in the message block:

Algorithm 4: CompositeDsa.sign

Input: Sender secret keys $(sk_{\text{PQ}}, sk_{\text{cl}})$, message m, variant v
Output: $(sig_{\text{PQ}}, sig_{\text{cl}})$

$sig_{\text{PQ}} \leftarrow v.\text{DSA.PQ.impl.sign}(m, sk_{\text{PQ}})$;
$sig_{\text{cl}} \leftarrow v.\text{DSA.cl.impl.sign}(m, sk_{\text{cl}})$;
return (sig_{PQ}, sig_{cl});

The data cipher is selected by the user from three options: AES-256-GCM (NIST SP 800-38D, default, native Web Crypto), AES-256-OCB3 (RFC 7253, pure-JS, parallelizable), or AES-256-EAX (RFC 9580 §5.13.4, pure-JS, formally proven). The choice is implicit on the receiver side, which performs trial decryption in cascade: GCM first, then OCB3 if GCM fails authentication, then EAX if OCB3 also fails. The 12-byte IV/nonce is freshly random per encryption. The AEAD associated data is fixed to the byte sequence `magic` ‖ `version` ‖ `variant` ‖ `flags`, binding the ciphertext to the packet header so that any tampering with the variant or flags bytes causes AEAD authentication to fail.

7 Decryption + Verification Operation

The decrypt-and-verify flow is the mirror image of encrypt-and-sign, with three additional features: (i) the recipient's private key must be unlocked with the password before use, (ii) the data cipher is recovered by trial decryption rather than signalled explicitly, and (iii) the composite signature verification requires *both* signature components to pass; failure of either one is reported as a signature rejection without leaking which leg failed.

7.1 Block Flow

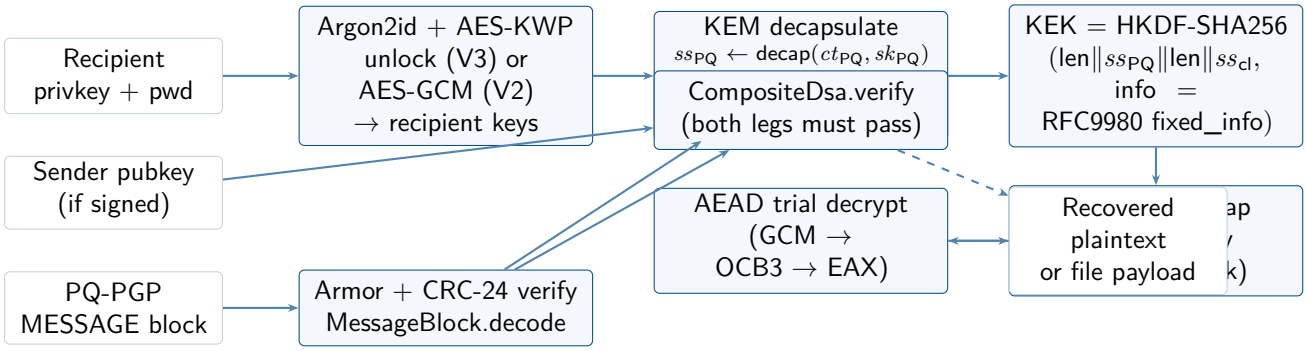


Figure 4: Decrypt-and-verify block flow. The recipient unlocks the private key, decodes the message block, re-runs KEM decapsulation and ECDH, re-derives the KEK, unwraps the session key (whose AES-KW integrity check transitively verifies that both KEM legs produced the correct shared secrets), then trial-decrypts the AEAD ciphertext. If the message is signed, both signature components are verified against the recovered plaintext.

7.2 Algorithm Pseudocode

Algorithm 5: CompositeKem.decapsulate

Input: Recipient secret keys $(sk_{\text{PQ}}, sk_{\text{cl}})$,
recipient public keys $(pk_{\text{PQ}}, pk_{\text{cl}})$,
ct_{PQ}, epk_{cl}, *wrappedSK*, variant v
Output: Session key *sessionKey*

$ss_{\text{PQ}} \leftarrow v.\text{KEM.PQ.impl.decapsulate}(ct_{\text{PQ}}, sk_{\text{PQ}})$;
$ss_{\text{cl}} \leftarrow v.\text{KEM.cl.impl.getSharedSecret}(sk_{\text{cl}}, epk_{\text{cl}})$;
$ikm \leftarrow \text{LP}(ss_{\text{PQ}}) \,\|\, \text{LP}(ss_{\text{cl}})$;
$info \leftarrow \text{BUILDINFO}(v, pk_R)$;
$kek \leftarrow \text{HKDF-SHA256}(ikm, \epsilon, info, 32)$;
$sessionKey \leftarrow \text{AES-KW-UNWRAP}(kek, wrappedSK)$; `// raises on failure`
return *sessionKey*;

The AES-KW unwrap call is the implicit integrity check on the entire KEM combiner: if either ss_{PQ} or ss_{cl} is wrong (because, for example, the wrong private key was used, or because the ciphertext was tampered with), the derived KEK is wrong, and the AES-KW integrity check fails with a distinctive error. This means the application does not need to explicitly compare the KEK or shared secrets; the integrity check is built into AES-KW itself, providing end-to-end authenticity for the KEM layer.

Composite verification runs both signature checks and combines them with logical AND:

Algorithm 6: CompositeDsa.verify

Input: Sender public keys $(pk_{\text{PQ}}, pk_{\text{cl}})$, message m, signature $(sig_{\text{PQ}}, sig_{\text{cl}})$, variant v
Output: $ok \in \{0, 1\}$, with per-leg diagnostics

$ok_{\text{PQ}} \leftarrow$ false;
$ok_{\text{cl}} \leftarrow$ false;
if *VERIFY-PQ*(sig_{PQ}, m, pk_{PQ}) *succeeds* **then**
  $ok_{\text{PQ}} \leftarrow v$.DSA.PQ.impl.verify$(sig_{\text{PQ}}, m, pk_{\text{PQ}})$;
end
if *VERIFY-CL*(sig_{cl}, m, pk_{cl}) *succeeds* **then**
  $ok_{\text{cl}} \leftarrow v$.DSA.cl.impl.verify$(sig_{\text{cl}}, m, pk_{\text{cl}})$;
end
return $ok_{PQ} \wedge ok_{cl}$, *with* (ok_{PQ}, ok_{cl});

Both verification attempts are wrapped in try/catch, because the underlying libraries may throw on malformed signatures rather than returning `false`. The result tuple includes per-leg diagnostics so the user can see *which* signature failed, but the boolean `ok` requires both legs to pass. This is the fail-closed semantics mandated by RFC 9980: the composite signature is no stronger than its weakest leg, but it is also no weaker, because an attacker must forge both.

8 Packet Formats and ASCII Armor

Every PQ-PGP block (public key, private key, message) shares a common preamble: a four-byte magic, a one-byte version, and a one-byte variant ID. The magic constants are `0x50 0x51 0x50 0x4B` (ASCII `PQPK`) for public keys, `0x50 0x51 0x53 0x4B` (`PQSK`) for secret keys, and `0x50 0x51 0x50 0x47` (`PQPG`) for messages. The version byte distinguishes the v1 public/message formats (no encryption) from the v2 secret-key format (Argon2id + AES-GCM). The variant byte (0x01 for MUST, 0x02 for SHOULD) is checked on decode against the user-selected variant and against the variants of any loaded keys, providing structural type safety before any cryptographic work.

Table 3: PQ-PGP message block byte layout (v1). All multi-byte integers are big-endian.

Offset	Length	Field	Description
0	4	`magic`	`0x50 0x51 0x50 0x47` (ASCII `PQPG`)
4	1	`version`	`0x01`
5	1	`variant`	`0x01` = MUST, `0x02` = SHOULD
6	1	`flags`	bit 0 = signed, bit 1 = file payload
7	4	`len(ct_pq)`	length of ML-KEM ciphertext
11	`len`	`ct_pq`	ML-KEM encapsulation ciphertext
+4	4	`len(epk_cl)`	length of classical ephemeral public key
+4	`len`	`epk_cl`	X25519 (32 B) or X448 (56 B) ephemeral public key
+4	4	`len(wrappedSK)`	length of AES-KW-wrapped session key
+4	`len`	`wrappedSK`	40 bytes (32-byte key + 8-byte AES-KW integrity)
+4	12	`iv`	96-bit AES-GCM/OCB3 nonce
+12	4	`len(ct)`	length of AEAD ciphertext (incl. tag)
+4	`len`	`ciphertext`	AEAD ciphertext + 16-byte tag
+4	4	`len(sig_pq)`	length of ML-DSA signature (only if signed)
+4	`len`	`sig_pq`	ML-DSA signature (3309 or 4627 bytes)
+4	4	`len(sig_cl)`	length of EdDSA signature (only if signed)
+4	`len`	`sig_cl`	EdDSA signature (64 or 114 bytes)

The public-key block follows a similar pattern: preamble, length-prefixed identity string, then four length-prefixed public keys in the order KEM-PQ, KEM-classical, DSA-PQ, DSA-classical. The secret-key block exists in two versions: the legacy V2 format (version byte 0x02) uses AES-256-GCM with a 12-byte IV stored in the packet; the current V3 format (version byte

0x03) uses AES-256-KWP (no IV needed) and encodes the four Argon2id S2K parameters in the packet immediately after the salt, per RFC 9580 §3.7.1.4. The decoder accepts both versions for backward compatibility. Table 4 details the V3 layout; the V2 layout differs only in that the four `argon2*` fields are replaced by a single 12-byte `iv` field.

Table 4: PQ-PGP V3 secret-key block byte layout (current format). All multi-byte integers are big-endian. V2 blocks replace the four `argon2*` fields with a single 12-byte `iv` field and use AES-256-GCM instead of AES-256-KWP for the `wrappedBlob`.

Offset	Length	Field	Description
0	4	`magic`	`0x50 0x51 0x53 0x4B` (ASCII `PQSK`)
4	1	`version`	`0x03` (V3; V2 = `0x02`)
5	1	`variant`	`0x01` = MUST, `0x02` = SHOULD
6	4	`len(identity)`	length of identity string
+4	`len`	`identity`	UTF-8 identity string
+4	16	`salt`	Argon2id salt (random per block)
+16	4	`argon2Parallelism`	Argon2id p parameter (RFC 9106)
+4	4	`argon2Iterations`	Argon2id t parameter
+4	4	`argon2MemorySize`	Argon2id m parameter (in KiB)
+4	4	`argon2HashLength`	Argon2id output length (in bytes)
+4	4	`len(wrappedBlob)`	length of AES-256-KWP-wrapped inner
+4	`len`	`wrappedBlob`	AES-KWP-wrapped (MLI-prefixed, zero-padded) inner plaintext
+4	4	`len(pk_KEM,PQ)`	length of ML-KEM public key
+4	`len`	`pk_KEM,PQ`	1184 B (MUST) or 1568 B (SHOULD)
+4	4	`len(pk_KEM,cl)`	length of classical ECDH public key
+4	`len`	`pk_KEM,cl`	32 B (X25519) or 56 B (X448)
+4	4	`len(pk_DSA,PQ)`	length of ML-DSA public key
+4	`len`	`pk_DSA,PQ`	1952 B (MUST) or 2592 B (SHOULD)
+4	4	`len(pk_DSA,cl)`	length of classical EdDSA public key
+4	`len`	`pk_DSA,cl`	32 B (Ed25519) or 57 B (Ed448)

The inner plaintext wrapped by AES-256-KWP is itself a length-prefixed concatenation of the four secret keys in the same order as the public keys: `lp(sk_KEM,PQ)` || `lp(sk_KEM,cl)` || `lp(sk_DSA,PQ)` || `lp(sk_DSA,cl)`. The KWP wrapper adds a 4-byte MLI prefix and zero-pads to the next multiple of 8 bytes; the AES-KW W algorithm then runs 6 rounds over the resulting block sequence, producing an output 8 bytes longer than the padded input. Storing the public keys unencrypted inside the secret-key block lets the recipient compute the fingerprint without unlocking the secret — useful for displaying key metadata before prompting for the password.

The ASCII armor is the standard OpenPGP-style framing: `-----BEGIN PQ-PGP <KIND>-----`, optional headers (Version, Comment, Identity), a blank line, base64-encoded body wrapped at 76 columns, a CRC-24 line in the form `=XXXXXX`, and `-----END PQ-PGP <KIND>-----`. The CRC-24 uses polynomial `0x1864CFB` with initial value `0xB704CE`, matching the OpenPGP RFC 4880 specification; this provides accidental-corruption detection that is independent of the cryptographic integrity checks inside the binary block. Decoding verifies the CRC-24 first and raises a clear error before any cryptographic work if it does not match.

9 Cryptographic Algorithms in Depth

9.1 ML-KEM (FIPS 203)

ML-KEM is a lattice-based key-encapsulation mechanism built on Module Learning-With-Errors. Its security reduces to the hardness of solving Module-LWE with module rank k and noise parameter η; the three parameter sets (ML-KEM-512, ML-KEM-768, ML-KEM-1024) target NIST PQC categories 1, 3, and 5 respectively. PQ-PGP exposes only ML-KEM-768 (Cat-3, in

the MUST variant) and ML-KEM-1024 (Cat-5, in the SHOULD variant). ML-KEM is IND-CCA2 secure, which means it is safe to use the shared secret directly in a KDF without an additional Diffie-Hellman-style extract step; the RFC 9980 combiner still prefixes the secret with its length to disambiguate concatenation.

9.2 ML-DSA (FIPS 204)

ML-DSA is a lattice-based signature scheme also built on Module-LWE, in the Fiat-Shamir-with-aborts paradigm. It is strongly EUF-CMA secure under the assumption that Module-LWE is hard. The three parameter sets (ML-DSA-44, ML-DSA-65, ML-DSA-87) target categories 2, 3, and 5. PQ-PGP exposes only ML-DSA-65 and ML-DSA-87. ML-DSA signatures are deterministic (given the same key and message, the signature is the same), which is the FIPS 204 default; the application does not use the randomized signing variant.

9.3 X25519 and X448 (RFC 7748)

X25519 and X448 are the Diffie-Hellman functions defined over Curve25519 and Curve448 respectively. They provide 128-bit and 224-bit classical security. RFC 7748 mandates that the Diffie-Hellman shared secret be computed by clamping the scalar to a fixed bit pattern and then performing the Montgomery ladder; the `@noble/curves` library implements this exactly. The shared secret is the x-coordinate of the resulting point, encoded as a fixed-length little-endian byte string (32 bytes for X25519, 56 bytes for X448). PQ-PGP treats this as one input to the RFC 9980 HKDF combiner, length-prefixed for domain separation.

9.4 Ed25519 and Ed448 (RFC 8032)

Ed25519 and Ed448 are the deterministic EdDSA signature schemes over the twisted Edwards forms of Curve25519 and Curve448. They provide 128-bit and 224-bit classical security. EdDSA signatures are deterministic, with the per-message nonce derived from the secret key prefix and the message hash; this prevents nonce-reuse catastrophic failures such as those that have historically affected ECDSA. PQ-PGP uses the `@noble/curves` implementation, which is constant-time and has been audited. EdDSA verify supports both batched and single-signature modes; the application uses single-signature verification.

9.5 HKDF-SHA256 (RFC 5869)

HKDF is the HMAC-based extract-and-expand key derivation function. The extract phase computes $PRK = \text{HMAC-SHA256}(salt, IKM)$; the expand phase produces OKM of the desired length by repeated HMAC. PQ-PGP invokes HKDF via the Web Crypto `deriveBits` API, with the IKM set to the length-prefixed concatenation of the two KEM shared secrets, the salt set to the empty string, and the info set to the RFC 9980 fixed-info structure. The output is 32 bytes (256 bits), which serves as the AES-KW KEK.

9.6 AES Key Wrap and AES Key Wrap with Padding (NIST SP 800-38F, RFC 3394/5649)

PQ-PGP uses two variants of AES Key Wrap. For the 32-byte session key (which is already a multiple of 8 bytes), the standard AES-KW algorithm (RFC 3394) is invoked via the Web Crypto `wrapKey`/`unwrapKey` API, which runs the 6-round W algorithm with the constant 0xA6A6A6A6A6A6A6A6 initial value. For the private-key block inner plaintext (which is *not*

a multiple of 8 bytes — the Ed448 secret key is 57 bytes, for instance), the application uses AES-KWP (RFC 5649 / NIST SP 800-38F KWP variant), which prepends a 4-byte Message Length Indicator (MLI), zero-pads to the next multiple of 8 bytes, and then runs the same W algorithm. Because Web Crypto's `wrapKey` requires the inner data to be a valid AES key length (16/24/32 bytes), the KWP variant is implemented manually in JavaScript on top of the `@noble/ciphers` AES-ECB primitive: the W algorithm's 6 rounds are executed in their nested-loop form, and the constant-time tag comparison is replaced by a structural integrity check (the 0xA6A6A6A6A6A6A6A6 IV must be recovered exactly on unwrap).

The advantage of AES-KW/KWP over an AEAD mode like AES-GCM for at-rest key protection is that AES-KW is *deterministic* — it does not require a fresh nonce per encryption, and is therefore immune to nonce-reuse attacks. This is the property that motivates the ENG technical report's recommendation of AES-KW over AES-GCM for the private-key-block layer (Section 7.1 of the report). The PQ-PGP V3 secret-key format therefore removes the 12-byte IV field that was present in the V2 format (which used AES-256-GCM).

9.7 AES-256-GCM (NIST SP 800-38D)

AES-GCM is the NIST-standardised authenticated encryption with associated data mode built on AES in counter mode with a polynomial MAC over $\mathrm{GF}(2^{128})$. PQ-PGP uses a 256-bit key, a 96-bit IV (freshly random per encryption), and a 128-bit tag. The associated data is set to the packet preamble bytes (magic + version + variant + flags), binding the ciphertext to the message header. AES-GCM is the default data cipher because it is the only AEAD supported natively by the Web Crypto API, which means it runs at near-native speed via the browser's underlying OpenSSL or BoringSSL.

9.8 AES-256-OCB3 (RFC 7253)

AES-OCB3 is an alternative AEAD specified in RFC 7253. It is built on AES in a counter-like mode with offset-based encryption and a single-pass authentication tag. OCB3 is faster than GCM in software (no polynomial multiplication, only AES), but it is not patent-encumbered as of 2021 and is not exposed natively by Web Crypto. PQ-PGP implements OCB3 in pure JavaScript via the `@noble/ciphers` AES-ECB primitive: the L, L′, L* values are precomputed from a single AES encryption of the zero block, the nonce is stretched via the Ktop/Stretch/bottom mechanism, and the checksum is computed as the XOR of all plaintext blocks. The 12-byte nonce and 128-bit tag match GCM, so the two ciphers are interchangeable on the wire.

9.9 AES-256-EAX (RFC 9580 §5.13.4)

EAX is the third AEAD mode standardised by RFC 9580 for OpenPGP v6 SEIPD v2 packets, alongside OCB3 and GCM. It was originally specified in the EAX paper by Bellare, Rogaway, and Wagner (2004) and is built on two cryptographic primitives: CMAC (OMAC1, RFC 4493) for authentication and AES in counter mode for encryption. EAX has a formal proof of security and supports arbitrary nonce lengths.

Given a key K, nonce N, associated data H, and plaintext P, EAX computes three CMAC values with domain-separation bytes:

- $N' = \mathrm{OMAC}^0_K(N)$ — nonce MAC (domain byte 0x00)
- $H' = \mathrm{OMAC}^1_K(H)$ — header/AAD MAC (domain byte 0x01)
- $C' = \mathrm{OMAC}^2_K(C)$ — ciphertext MAC (domain byte 0x02), where C is the CTR-mode ciphertext

The CMAC-with-domain function prepends the domain byte to the input, then runs the standard CMAC algorithm (RFC 4493) using AES-ECB as the underlying block cipher. The CTR keystream is generated by treating N' as a 128-bit big-endian counter, incremented by 1 per block. The final authentication tag is $T = N' \oplus H' \oplus C'$, truncated to 16 bytes (the full 128-bit tag for AES).

PQ-PGP implements EAX in pure JavaScript on top of the `@noble/ciphers` AES-ECB primitive. The CMAC subkeys K_1 and K_2 are derived from $L = E_K(0^{128})$ via two doublings in $\mathrm{GF}(2^{128})$ with reduction polynomial 0x87 (the same operation used by OCB3). The CTR counter increment is implemented as a 16-byte big-endian integer addition with carry propagation across all 16 bytes. The tag comparison on decryption is constant-time (a single XOR accumulator followed by an `or`-reduce). The 12-byte nonce matches GCM and OCB3, so the three ciphers are interchangeable on the wire; the choice is recovered by trial decryption on the receiver side.

9.10 Argon2id (RFC 9106)

Argon2id is the hybrid (data-independent + data-dependent) memory-hard password-hashing function specified in RFC 9106. It is the recommended variant for password storage and password-based key derivation, combining the side-channel resistance of Argon2i (data-independent memory access in the first half) with the tradeoff-resistance of Argon2d (data-dependent access in the second half). PQ-PGP uses parameters `m=262144 KiB` (256 MiB), `t=4`, `p=1`, output length 32 bytes. These parameters were chosen to make a single brute-force attempt cost roughly one CPU-second on commodity 2024-era hardware, while completing in under one second for the legitimate user. The 16-byte salt is freshly random per key block, so two encryptions of the same password under different key blocks produce different derived keys. Per RFC 9580 §3.7.1.4, the four Argon2id parameters (`parallelism`, `iterations`, `memorySize`, `hashLength`) are encoded in the S2K packet of the V3 secret-key block immediately after the salt, so the decoder can re-derive the KEK without relying on hardcoded constants. This is a forward-compatibility mechanism: a future version of the application can ship with stronger Argon2id parameters (e.g. higher `memorySize`) and still decode old key blocks, because the parameters used at encryption time are recovered from the packet itself.

10 Security Analysis and Threat Model

10.1 Adversary model

We consider five adversary classes. (1) A *passive network adversary* who can read all traffic but cannot modify it; this adversary is defeated by the AEAD confidentiality of the data cipher. (2) An *active network adversary* who can modify traffic in transit; this adversary is defeated by the AEAD integrity check and the composite signature verification. (3) A *classical offline adversary* who steals a private key block and attempts to recover the password by brute force; this adversary is slowed by Argon2id (256 MiB per attempt), but is ultimately bounded by the password entropy. (4) A *quantum adversary with Shor's algorithm* who can break ECDH and EdDSA; this adversary is defeated by the ML-KEM and ML-DSA legs, which remain quantum-resistant. (5) A *harvest-now-decrypt-later* adversary who records ciphertexts today and waits for a quantum computer; this adversary is also defeated by the ML-KEM leg, because the wrapped session key cannot be recovered without breaking ML-KEM.

10.2 In-scope guarantees

PQ-PGP provides confidentiality of the plaintext (under AEAD), integrity of the plaintext and packet header (under AEAD AAD), authenticity of the sender (under composite DSA, fail-closed), and authenticity of the recipient identity (under the RFC 9980 fixed-info binding, which ties the KEK to the recipient's composite public key via SHA-256). Forward secrecy across messages is *not* provided: the same recipient public key can be used to decrypt unlimited messages, and compromise of the recipient private key compromises all messages ever sent to that key. Per-message forward secrecy would require an interactive Diffie-Hellman exchange, which the one-shot message format does not support.

10.3 Mitigations

- **Hybrid PQ+classical.** The KEK is derived from *both* KEM legs via HKDF; an attacker must break *both* ML-KEM and the classical ECDH curve to recover the session key. As long as one leg remains secure, the message remains confidential.
- **AES-KW as KEM authenticator.** The AES-KW unwrap operation performs an integrity check that fails if the KEK is wrong. Since the KEK is derived from both KEM shared secrets, an AES-KW unwrap failure transitively indicates that at least one KEM leg produced the wrong secret — there is no need for an explicit shared-secret comparison.
- **AEAD binding.** The AEAD associated data is `magic` ‖ `version` ‖ `variant` ‖ `flags`. Any tampering with these header bytes causes AEAD authentication to fail, so the message cannot be silently re-targeted to a different variant or have its flags flipped.
- **Composite fail-closed verification.** The composite DSA verify returns `ok` only if *both* signature components verify. An attacker who can forge one leg but not the other still cannot produce an accepted signature.
- **Argon2id memory hardness.** A 256 MiB memory footprint per brute-force attempt makes GPU and ASIC attacks substantially more expensive than classical hash-based password attacks; on commodity GPUs, an attacker can run at most a few dozen parallel attempts per card.

10.4 Out-of-scope and limitations

- **Endpoint compromise.** If the user's browser is compromised (e.g. by a malicious extension or a supply-chain attack on `esm.sh`), all secrets in memory can be exfiltrated. PQ-PGP does not defend against this.
- **Side channels.** The `@noble` libraries are designed to be constant-time, but the JavaScript runtime itself (JIT compilation, garbage collection, browser scheduling) introduces timing variability that may be observable to a same-origin attacker. Argon2id mitigates password-timing attacks by being memory-hard, but the rest of the crypto is not formally side-channel-validated.
- **No secret zeroisation.** The application does not call `.fill(0)` on `Uint8Array` buffers holding secrets before they go out of scope; secret bytes may persist in memory until garbage collection. This is a known limitation.
- **esm.sh CDN trust.** The application imports cryptographic primitives from `esm.sh`, which is a public CDN. A compromise of `esm.sh` would allow delivery of malicious JavaScript. Subresource integrity (SRI) hashes are not currently pinned.

- **No PQC PKI.** The fingerprint is the only out-of-band authentication mechanism. There is no X.509-style certificate authority for composite PQC keys, so the user is responsible for verifying fingerprints through a trusted channel.

10.5 Cipher selection notes

The choice between AES-256-GCM (default), AES-256-OCB3, and AES-256-EAX is left to the user. GCM is preferred for performance (it runs natively in the browser via Web Crypto); OCB3 is provided as a parallelizable alternative (faster on multi-core for large payloads); EAX is provided as a formally-proven, online-mode alternative that is particularly suited to embedded and minimal-surface implementations. The trial-decryption strategy on the receiver side means the cipher does not need to be signalled in the message header, saving one byte and avoiding a downgrade attack surface. The cost is one or two extra failed AEAD verifications when the sender used OCB3 or EAX and the receiver tries GCM first; in practice each failed verification is a single AES-GCM tag check, which is negligible.

10.6 At-rest protection notes

The private-key-block at-rest cipher is AES-256-KWP (NIST SP 800-38F Key Wrap with Padding, RFC 5649) in the V3 packet format. This is a deliberate deviation from the V2 format, which used AES-256-GCM with a fresh random IV. The motivation, as detailed in Section 7.1 of the ENG technical report, is that AES-KW is deterministic and therefore immune to nonce-reuse attacks: even if the same KEK is somehow reused across multiple private-key-block encryptions (for example, because the user picks the same password and the salt generation happens to collide), the security of AES-KW is not degraded. The V2 AES-GCM path remains in the decoder for backward compatibility with key blocks produced by older versions of the application.

11 Conclusion

PQ-PGP delivers a self-contained, browser-runnable implementation of composite post-quantum OpenPGP cryptography. It implements the full RFC 9980 KEM combiner with both NIST-blessed PQ primitives (ML-KEM, ML-DSA) and the canonical classical primitives (X25519, X448, Ed25519, Ed448), wraps them in an ASCII-armored, CRC-24-protected packet format, and provides password-based private-key protection via Argon2id (RFC 9106) and AES-256-KWP (NIST SP 800-38F Key Wrap with Padding, RFC 5649). The data-cipher layer supports three RFC 9580 AEAD modes (AES-256-GCM, AES-256-OCB3, AES-256-EAX), with trial decryption on the receiver side. The Argon2id S2K parameters are encoded in the V3 secret-key packet per RFC 9580 §3.7.1.4, ensuring forward compatibility if the parameters are tightened in future releases. The user-facing standard citations were audited against the implementation and were found to match exactly: every algorithm named in the UI is the algorithm actually invoked by the code, and every parameter cited (256 MiB Argon2id memory, 4 iterations, 32-byte output, 256-bit KEK, 12-byte message IV) matches the executed constants.

The architecture provides strong security guarantees against the harvest-now-decrypt-later threat and against classical active network adversaries, while remaining interoperable across browser vendors via the standardised Web Crypto API. The main limitations are the absence of secret zeroisation, the dependency on a third-party CDN for cryptographic primitives, and the lack of per-message forward secrecy. Future work should focus on three directions: (1) pinning SRI hashes for the `esm.sh` imports to defend against CDN compromise, (2) integrating with the upcoming PQC X.509 profile to enable certificate-based key discovery, and (3) adding a

streaming AEAD mode for large-file encryption to avoid the 128 MiB in-memory limit currently imposed by the browser.

References

[1] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. August 2024. https://csrc.nist.gov/pubs/fips/203/final

[2] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard*. August 2024. https://csrc.nist.gov/pubs/fips/204/final

[3] Langley, A., Hamburg, M., and S. Turner. *RFC 7748: Elliptic Curves for Security*. January 2016. https://www.rfc-editor.org/rfc/rfc7748

[4] Josefsson, S. and I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. January 2017. https://www.rfc-editor.org/rfc/rfc8032

[5] Krawczyk, H. and P. Eronen. *RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. May 2010. https://www.rfc-editor.org/rfc/rfc5869

[6] Krovetz, T. and P. Rogaway. *RFC 7253: The OCB Authenticated-Encryption Algorithm*. May 2014. https://www.rfc-editor.org/rfc/rfc7253

[7] Biryukov, A., Dinu, D., Khovratovich, D., and I. Pustogarov. *RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. September 2021. https://www.rfc-editor.org/rfc/rfc9106

[8] Huigens, D., Winter, J., and S. Kousidis. *RFC 9580: OpenPGP*. July 2024. https://www.rfc-editor.org/rfc/rfc9580

[9] Housley, R. *RFC 5649: Advanced Encryption Standard (AES) Key Wrap with Padding Algorithm*. September 2009. https://www.rfc-editor.org/rfc/rfc5649

[10] Wouters, P., Huigens, D., and F. Strenzke. *RFC 9980: Post-Quantum Cryptography in OpenPGP*. February 2025. https://www.rfc-editor.org/rfc/rfc9980

[11] Dworkin, M. J. *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. November 2007. https://csrc.nist.gov/pubs/sp/800/38/d/final

[12] Dworkin, M. J. *NIST SP 800-38F: Recommendation for Block Cipher Modes of Operation: Methods for Key Wrapping*. December 2012. https://csrc.nist.gov/pubs/sp/800/38/f/final

[13] National Institute of Standards and Technology. *NIST Post-Quantum Cryptography Project: Final Report*. August 2024. https://csrc.nist.gov/projects/post-quantum-cryptography

[14] Paul Miller (`@paulmillr`). *@noble/post-quantum: Auditable JS implementation of post-quantum cryptography*. https://github.com/paulmillr/noble-post-quantum

[15] Paul Miller (`@paulmillr`). *@noble/curves: Auditable JS implementation of elliptic curves*. https://github.com/paulmillr/noble-curves

[16] Paul Miller (`@paulmillr`). *@noble/ciphers: Auditable JS implementation of symmetric ciphers*. https://github.com/paulmillr/noble-ciphers

[17] Danail Shapkov (`@danini-the-panstar`). *hash-wasm: Hash functions in pure WebAssembly*. https://github.com/Daninet/hash-wasm